

Övning 9: Minigaraget med React

Introduktion

I denna uppgift ska du bygga en modern, interaktiv fordonskatalog för ett digitalt garage i **React och TypeScript**, med hjälp av **Tailwind CSS** för gränssnittet.

Eftersom du har vana av C# och TypeScript sedan tidigare ligger fokus i denna uppgift helt på **Reacts komponentminne (useState)**, **immutabilitet med spread-operatörn (...)**, samt **listrendering med .map() och .filter()**.

Nivå 1: Grundläggande funktioner (MVP)

Målet med nivå 1 är att få till det grundläggande flödet där du kan lägga till och ta bort bilar i garaget.

Funktionskrav för Nivå 1:

1. **Statistikräknare:** Högst upp i appen ska det finnas en text som visar det aktuella antalet bilar i garaget (t.ex. *"Antal bilar i garaget: 2"*). Om garaget är tomt ska en text med *"Garaget är helt tomt"* visas.
2. **Formulär för parkering:**
 - a. Ett inmatningsfält för **Regnummer** (t.ex. ABC123).
 - b. Ett inmatningsfält för **Märke** (t.ex. Volvo).
 - c. En knapp med texten **"Parkera bil"**.
3. **Immutabilitet vid tillägg:** När man klickar på knappen ska ett nytt bilobjekt (med unikt ID via `Date.now()`) läggas till i state-arrayen med hjälp av spread-operatörn (`[...prev, newCar]`). Formuläret ska därefter nollställas.
4. **Listrendering och borttagning:**
 - a. Rendera ut alla parkerade bilar i listan med `.map()`.
 - b. Varje bil-kort ska ha en **"Ta bort"**-knapp som använder `.filter()` för att ta bort just det fordonet från statet.

Nivå 2: Smart validering av regnummer (JavaScript-logik)

När nivå 1 fungerar är det dags att höja kvaliteten på inmatningen. I denna nivå bygger vi in regler för hur ett svenskt registreringsnummer får se ut.

Funktionskrav för Nivå 2:

1. **Regel för formatet:** Ett giltigt svenskt regnummer i detta exempel ska bestå av **3 bokstäver följt av 3 siffror** (standardformat, t.ex. ABC123) eller **3 bokstäver följt av 2 siffror och 1 bokstav** (t.ex. ABC12A) . Små bokstäver ska automatiskt konverteras till versaler.
2. **Reguljära uttryck (Regex):** Använd JavaScripts Regex för att validera strängen.
3. **Felhantering i gränssnittet:**
 - a. Om användaren försöker spara med tomma fält, eller om regnumret **inte** matchar formatet AAA111 eller AAA11A, ska ett snyggt rött felmeddelande visas ovanför formuläret (använd inte `alert()`).
 - b. Bilen får endast sparas om valideringen passerar utan anmärkning.

Tips på vägen

- Det finns ett start-skäl för dem som behöver hjälp på vägen, "Övning 9 Minigaraget Startskal.tsx". Ersätt innehållet i App.tsx från denna.
- Kom ihåg att importera `useState` från 'react' .
- Kontrollera att Tailwind CSS är inkopplat i din `index.html` så att designen renderas korrekt.

```
<!-- Lägg till denna rad -->  
<script src="https://cdn.tailwindcss.com"></script>
```

- Testa din validering i konsolen (`console.log`) om du sätter fast dig på Regex-mönstret!

Exempel på utseende

Minigaraget (Slutlösning)

Antal bilar i garaget: 2

Parkera nytt fordon

Regnummer

T.EX. ABC123

Märke

t.ex. Volvo

Parkera bil

Parkerade fordon

ABC123	Volvo	Ta bort
XYZ789	Saab	Ta bort